

Team and Project Plan

PORTFOLIO TASK 1

Unit code: COS40005

Unit Name: Computing Technology Project A

Submission date:29/3/2026

Student Name	Student Id	Statement of contribution to the report
Umer Naseer	104404127	33.33
Frandy Meidy Fabiola	102765149	33.33
Cindy Wong Ing Ying	102779425	33.33

ACKNOWLEDGMENT OF COUNTRY

Each team member identifies:

the Traditional Owners of the land they lived on while completing this work (if living in Australia).

Table of Contents

ACKNOWLEDGMENT OF COUNTRY	0
PART 1 Team Code of Conduct	2
1. TEAM PROFILE	2
2. TEAM ROLE	3
3. TEAMWORK ROADMAP	4
4. DOCUMENT MANAGEMENT	5
5. RISK MITIGATION	6
PART 2 Project overview and plan	7
1. PROBLEM STATEMENT	7
2. SCOPE	7
3. STAKEHOLDER	8
4. HIGH-LEVEL REQUIREMENTS	8
5. SOLUTION APPROACH	8
High level architecture summary.....	11
6. PRODUCT BACKLOG	12
7. QUALITY PLAN	16
Quality expectations	16
Measurable targets.....	17
Steps to assure quality	18
Final note on quality planning	19
8. RESEARCH AND INVESTIGATIONS	20
Research on Existing Systems.....	21
Specialized Research Topics	22
Synthesis and Justification	22
9. KOST ANALYSIS	23
Reference list	25

PART 1 Team Code of Conduct

1. TEAM PROFILE

I am Umer Naseer, and I bring both technical and practical problem-solving skills to the team. I have experience with HTML, CSS, JavaScript, SQL, PHP, and some frontend and backend development, and I am also learning newer tools and frameworks needed for this project. I am comfortable adapting and picking things up quickly. In teamwork, I am direct and honest in communication, which helps when discussing ideas, problems, and progress clearly. I prefer keeping things straightforward so the team can move faster and avoid confusion. I am also comfortable taking initiative, speaking with clients or supervisors, and helping the team stay focused and organized throughout the project.

My name is Frandya Meidy Fabiola, and I have technical knowledge in programming languages such as **HTML, CSS, PHP, C#, SQL, and python**, particularly for both back and front-end development. I can take any available task, and I can communicate in English. I try to finish the task given to me and attempt to be available to do the project as much as possible. I also try to learn new tools needed to perform the tasks. I also want to get direct feedback regarding my work and in the team to avoid awkwardness.

I am Cindy Wong Ing Ying, a team member with technical skills in **HTML, CSS, React Native, C#, C, Python, MySQL, and PHP**, allowing me to contribute to both frontend and basic backend development. I am comfortable building user interfaces and mobile app features. I can communicate in **Bahasa Malaysia, Chinese, and English**, which helps me work well with different people. In terms of soft skills, I am responsible and able to complete tasks on time. My communication style is calm and cooperative rather than outspoken, so I listen well and support my teammates. In teamwork, I act as a **contributor**, helping with development tasks and assisting others when needed.

Student name	Technical skills	Soft skills	Communication	Teamwork	Other
Umer Naseer	HTML, CSS, JavaScript, SQL, PHP.	Practical, adaptable, takes initiative, stays focused on tasks	Direct and clear, comfortable discussion with client and supervisor	Team leader, coordinates work, supports team progress	Comfortable with planning, documentation, and database related tasks
Frandy Meidy Fabiola	HTML, CSS, C#, SQL, PHP, python	Complete Task given, available	Communicate as needed, open to feedback	Contributor, provide support as needed,	Comfortable with data related tasks and willing to learn new tools.
Cindy Wong Ing Ying	HTML, CSS, React Native, C#, C, Python, MySQL, PHP	Responsible, able to complete tasks on time	Calm, cooperative, can speak Bahasa Malaysia, Chinese, English	Contributor, supports team, assists others	Able to build UI and mobile app features

Table 1: Team Profile

2. TEAM ROLE

Student name	Team role	Justification
Umer Naseer	Team Leader / Client Liaison / Database Developer	suitable for this role because he can handle client communication, planning, and database related work. I have experience in SQL, web development, and problem solving, which helps in leading the team and supporting the schema comparison and migration parts of the project.
Frandy Meidy Fabiola	Backend Developer / Database Developer	Have skill with HTML, CSS, C#, SQL, PHP, and python can take any available task able to learn new tools needed to perform the tasks.
Cindy Wong Ing Ying	Frontend Developer / UI Developer	I have skills in HTML, CSS, React Native, C#, C, Python, MySQL and PHP which help me handle frontend and some backend tasks. I can build UI and mobile features, so this role fits my technical abilities and allows me to contribute effectively to the project.

Table 2: Team Role

3. TEAMWORK ROADMAP

Our team will have a weekly meeting either physically or through Google Meet, depending on availability. During meetings, we will discuss progress, problems, and plan the next tasks. For daily communication, we use WhatsApp for quick updates.

Tasks will be estimated based on difficulty and time required, then assigned according to each member's skills. Progress will be monitored during weekly meetings and through WhatsApp updates. We use GitHub to store code, track changes, and collaborate on development, while Google Drive is used for documents and files.

For team rules, all members should communicate clearly, respect each other, and complete tasks on time. If any issues or delays happen, members must inform the team early. If plans do not go as expected, we will redistribute tasks or adjust deadlines to ensure the project stays on track.

Area	Plan
Weekly team meeting	Once a week, usually early in the week
Meeting mode	Google Meet, Microsoft Teams, or physical meeting if possible
Informal communication	WhatsApp
Formal communication	Email
Response time	Preferably within the same day, faster for urgent matters
Internal team deadline	At least one day before actual submission
Revision process	One teammate reviews another member's work before finalising
Task estimation	Small, medium, and large based on effort and difficulty
Task assignment	Based on team role, workload, and suitability
Task monitoring	Trello task board
Code storage	GitHub
Document storage	Google Drive

Table 3: Team workflow and guideline

4. DOCUMENT MANAGEMENT

Our team will manage documents using a few simple and effective tools to keep everything organized. We use Microsoft Word to prepare, edit, and update the project report, ensuring that the latest version is always saved to avoid confusion or data loss. All team members will make sure to update the document regularly, so everyone is working on the most current version.

For communication, we use WhatsApp to discuss progress, share updates, and quickly clarify any issues among team members. This helps improve teamwork and ensures fast responses when needed. We also use Notepad during meetings to record important notes, especially client requirements and key discussion points, so nothing is missed.

For coding and technical work, we use GitHub to store our code, track changes, and manage different versions. This allows multiple team members to work on the project at the same time without overwriting each other's work. It also helps us keep a clear history of changes and makes collaboration more efficient throughout the project.

5. RISK MITIGATION

<i>Risk</i>	<i>Impact</i>	<i>Mitigation Plan</i>
Lack of technical skills in some areas	Tasks may take longer or produce errors	Team members will learn together, share knowledge, and use online resources or tutorials
Team member unavailable (sick/busy)	Delay in task completion and affect progress	Inform early, redistribute tasks among team, adjust timeline if needed
Poor communication	Misunderstanding and mistakes in work	Use WhatsApp actively, update regularly, and confirm tasks clearly
Client requirements unclear	Wrong development direction	Take clear notes, confirm with clients during meetings, and ask questions early

Table 4: Risk mitigation

PART 2 Project overview and plan

1. PROBLEM STATEMENT

In modern software development, database schemas frequently change as systems continue to evolve and new features are added. However, managing these changes across different environments such as development, staging, and production is often done manually. This manual process can lead to various issues, including inconsistencies between environments, version conflicts, missing updates, and deployment errors. As a result, teams may struggle to maintain accurate, synchronized, and up-to-date databases, which can affect system performance and reliability.

Furthermore, without proper version control and tracking, it becomes difficult to monitor what changes have been made, who made them, and when they were applied. This increases the risk of errors and makes it harder to fix problems when they occur. Therefore, this project aims to develop a database schema comparison and version control system that can automatically detect differences between schemas, manage schema versions efficiently, and generate SQL migration scripts for controlled updates. The main objective is to improve efficiency, reduce human errors, enhance collaboration among team members, and provide a more reliable and structured way of handling database changes across different environments.

2. SCOPE

The scope of this project is to develop a system that can compare database schemas by finding differences in tables, columns, indexes, and relationships. The system will also track schema versions, generate SQL scripts (including rollback scripts), and provide simple query analysis and performance suggestions. It will include role-based access control to manage user permissions and integrate with Git to store approved scripts and track changes.

However, this project will not include designing a database from the beginning. It will mainly focus on PostgreSQL and may not support all types of databases. It also will not automatically deploy changes to production without user approval, as users must review changes first. Features not related to database management are also not included in this project.

3. STAKEHOLDER

The main stakeholders of this project include the client, Jaden Choo Wei Chiat, who provides the project requirements, feedback, and overall direction. The supervisor, Jason Thomas Chew, is responsible for guiding the project, giving advice, and evaluating the final outcome. The student developer is responsible for designing, developing, and testing the system to meet the project goals.

End users such as software developers and database administrators (DBAs) will use the system to compare schemas, manage versions, and handle database changes more efficiently. In addition, the application support team may be involved in reviewing the system, testing its usability, and deciding whether it is suitable for future use or implementation in real-world environments.

4. HIGH-LEVEL REQUIREMENTS

This project is a new prototype system designed as a standalone tool that connects with external components such as PostgreSQL databases, GitHub for version control, and Microsoft authentication services. The system will provide main functional features like schema comparison, version tracking, SQL migration and rollback script generation, and basic query performance analysis to help users manage database changes more efficiently.

For non-functional requirements, the system must ensure security through proper authentication and access control. It should also have good performance and fast response time, be easy to use with a clear interface, and be reliable when handling schema comparisons and updates.

From a technical side, the system will be developed using Next.js as the main framework, PostgreSQL with Sequelize for database management, and Tailwind CSS for styling the interface. Docker will be used for deployment to ensure consistency across environments, and Jest will be used for testing to maintain system quality and reduce errors.

5. SOLUTION APPROACH

Our proposed solution is a web-based system for comparing database schemas across environments, tracking schema versions, and managing migration scripts in a safer and more organized way. The main issue behind the project is that database changes made in development are not always recorded properly before moving to stage or production. Because of that, teams can end up with missing columns, wrong column types, missing tables, or other schema mismatches that only show up later when the live system is already being used. Based on the project description and the client discussion, we see the project as mainly centered around two core areas, which are schema comparison and migration version control.

At a high level, we are planning to use layered architecture because it is easier for us to design, build, and explain. The system will have a frontend layer, an application layer, and a data layer. The front end will be responsible for the user interface, the application layer will handle the main processing and decision making, and the data layer will handle both the system's own stored records and the external databases being checked. We think this is the most practical approach for a student team because it keeps things structured without making the design too complicated too early.

For the frontend, we plan to use Next.js. This choice is mainly because it was the framework requested in the client's stack guidance, but also because it gives us a clean way to build the user interface together with server-side logic in one project structure. The frontend will act as the main point of interaction for users and will support the overall flow of authentication, connection management, schema comparison, migration review, and version tracking. Since the intended users are mainly IT staff, including some who are not deeply confident with database details, the interface needs to make the process clearer and more manageable than doing everything manually.

The main application logic will sit behind that frontend layer. This part of the system will handle authentication, validation, schema comparison processing, version checks, migration SQL generation, and GitHub integration. Instead of directly making uncontrolled changes to a target database, the application logic will focus on detecting differences, preparing the required SQL, and supporting a review-based workflow. That is an important design choice for us because the client's main concern is not just comparison itself, but making schema changes more visible, traceable, and controlled before anything is applied.

For the system's own stored data, we plan to use PostgreSQL together with Sequelize ORM. This part will likely be used for users, saved database connections, comparison history, approval records, migration history, and related application data. For the actual schema inspection work, though, we expect to rely on raw SQL queries against PostgreSQL metadata rather than ORM alone. The reason is simple. Schema comparison is too specific and too tied to database internals to depend fully on ORM abstraction. We will likely need to query system metadata directly to inspect tables, columns, constraints, indexes, and version related information properly.

Authentication is planned through Microsoft account login using NextAuth, which follows the stack direction given by the client. This makes sense for the kind of user expected to use the system and keeps the design closer to something that could be adopted more seriously later if needed. Docker is planned for containerised setup and deployment, mainly to keep development and demonstration environments more consistent. Jest will be used later for testing, especially for main pages and user facing functionality.

In terms of overall flow, architecture is meant to support a process where a user can work from authenticated access into connection management, then into schema comparison, and finally into migration handling and version tracking. The important point here is not the exact screen by screen behavior yet, but the fact that the system is being designed around workflow. The application should help users inspect differences between environments, understand what is out of sync, and move toward a safer migration process instead of relying on memory or manual tracking.

We also looked at a few alternative approaches before settling in this direction. One option was to use a more familiar stack such as plain React for the frontend with a separate backend using Express or Flask. That would probably have been more comfortable for us at the start, especially since some parts of the required stack are still new to us. Still, we decided not to go in that direction because the application support guidance specifically points toward Next.js, PostgreSQL, Sequelize, NextAuth, Docker, and Jest. Also, even though the client said the project is not strictly intended for production right now, they also mentioned that a strong enough result could still be considered for future use. Because of that, it makes more sense for us to stay close to the requested stack rather than choosing something easier but less aligned.

Another approach we considered was trying to build the full feature set from the beginning, including advanced reporting, query execution analysis, performance suggestions, monitoring

UNIT CODE COS40005
PORTFOLIO TASK 1

style features, and AI supported explanation. We did not choose that as our main direction because it would be too much too early. Even the core comparison and version control parts already involve a lot of technical work, and forcing all the advanced features in from the start would raise the risk of ending up with a large but weak system. Our current plan is to get the core architecture and workflow working first, then improve it later if time and progress allow.

We also thought about whether the system should directly apply detected changes to the target database once differences are found. We rejected that idea because it goes against the safer workflow the client seems to want. A review first model makes more sense here. In our approach, the system supports the process by generating and organising the migration scripts, while leaving actual acceptance and later execution under control. That gives better traceability and lowers the chance of reckless changes.

At this stage, we think this approach gives us a solid direction without pretending that every lower-level detail is already fixed. Some parts will likely need adjustment once we begin implementation. For example, the exact rules for classifying major, minor, and patch changes may need refinement after testing with real schemas. The same goes for version tracking logic, GitHub integration depth, and rollback handling. So, this is our current high level architecture plan, but we expect to refine parts of it as we learn more during development.

High level architecture summary

User

Next.js frontend

Next.js server-side logic and API routes

Microsoft authentication with NextAuth

Internal application database using PostgreSQL and Sequelize

External target databases for schema inspection and comparison

GitHub for storing approved migration scripts

6. PRODUCT BACKLOG

Our backlog is based on the client's requirements, the project description, and our current understanding after the meeting. We have tried to make it detailed enough to guide development, but we also know the backlog may change as we learn more and as difficulty becomes clearer. So, these sprint allocations are tentative, not final promises.

For business value, let say we try to use High, Medium, and Low for easier understanding. And, we are going to add an ID, to look back for future reference.

ID	Backlog Item	Business Value	Dependency	Tentative Sprint
1	Set up project repo, branch workflow, and team development process	High	None	Sprint 1
2	Set up Next.js project structure	High	None	Sprint 1
3	Set up PostgreSQL and Sequelize for application data	High	2	Sprint 1
4	Set up Docker development environment	Medium	2, 3	Sprint 1
5	Configure Microsoft login using NextAuth	High	2	Sprint 1
6	Create role-based access control for viewer and deployment roles	High	5, 3	Sprint 1

7	Create UI for database connection entry and management	High	2, 3	Sprint 1
8	Add connection testing and host or port validation	High	7	Sprint 1
9	Build schema selection page for comparing two databases or environments	High	7	Sprint 2
10	Read PostgreSQL schema metadata using raw SQL queries	High	8, 9	Sprint 2
11	Compare tables and detect missing tables	High	10	Sprint 2
12	Compare columns, data types, and nullability	High	10	Sprint 2
13	Compare indexes, foreign keys, and constraints	High	10	Sprint 2
14	Generate comparison summary showing A versus B and B versus A	High	11, 12, 13	Sprint 2
15	Highlight critical versus minor schema differences	Medium	14	Sprint 2
16	Create migration SQL generation for missing tables and columns	High	14	Sprint 3
17	Allow user review and edit of generated SQL	High	16	Sprint 3

18	Save accepted migration scripts as versioned files	High	17	Sprint 3
19	Design version naming logic using major, minor, and patch	High	18	Sprint 3
20	Create version tracking table in application database	High	19	Sprint 3
21	Detect version table in target schema	High	10	Sprint 3
22	Compare database versions against available migration scripts	High	20, 21	Sprint 3
23	Show pending migration versions for a selected database	High	22	Sprint 3
24	Integrate GitHub push for approved migration scripts	High	18	Sprint 3
25	Pull migration scripts from GitHub for deployment view	Medium	24	Sprint 4 (Sem2)
26	Allow controlled execution of approved scripts into target database	High	23, 25	Sprint 4
27	Show deployment preview before execution	High	26	Sprint 4
28	Maintain migration execution history and audit log	High	26	Sprint 4
29	Basic rollback support for generated scripts	Medium	26	Sprint 5

30	Save comparison history and templates	Medium	14	Sprint 5
31	Add Jest testing for main webpages	High	Core pages completed	Sprint 5
32	Improve UI polish and error handling	Medium	Ongoing	Sprint 5
33	Query execution analysis using EXPLAIN	Medium	Core comparison stable	Sprint 6
34	Show readable explanation of execution plan	Medium	33	Sprint 6
35	Detect common SQL anti patterns	Medium	33	Sprint 6
36	Performance suggestion module	Low	33, 35	Sprint 6
37	Performance dashboard and monitoring trends	Low	36	Sprint 6
38	AI assistance or suggestions if time allows us	Low	Core modules stable	Sprint 6

Table 5: Product Backlog

Planned releases:

Alpha release at the end of Semester 1

For Alpha, we expect to complete the core workflow that the client seemed most concerned about. This means login, role-based access, database connection entry, schema selection, schema comparison, difference summary, and initial SQL generation. If possible, we also want a basic versioning concept by this point. Our goal for Alpha is to prove that the system can compare schemas and produce a useful output, even if the full migration control flow is still incomplete.

UNIT CODE COS40005
PORTFOLIO TASK 1

Beta release at the end of Semester 2

For Beta, we expect a more complete system. That should include version tracking, GitHub storage of approved scripts, pending migration detection, controlled execution flow, execution history, some rollback support, and testing. Advanced items like query analysis, performance suggestions, and AI help are more likely to be Beta extras if we have enough time and if the core flow is already solid.

We also would like to be realistic. If some features turn out a bit harder than expected, especially GitHub automation, rollback, or version comparison logic, we may need to reduce some scope or move some lower priority features to later sprints. We think that is better than just pretending everything will fit. So, this plan might change as time passes.

7. QUALITY PLAN

Our quality goal is not just to make the system work once on our laptops. We want the deliverables to be correct, understandable, reasonably safe, and usable for the kind of technical users the client described. Since the project deals with database schemas and migration scripts, quality matters a lot because even a small mistake can lead to wrong changes being suggested or applied.

Quality expectations

First, the system should be functioning for the core workflow. If a user compares two schemas, the system should correctly identify missing tables, missing columns, and column type mismatches in the cases we support. We do not want the tool to give a false impression that two schemas match when they do not. For the early prototype (by wk6), our target is that the main comparison flow works on our planned test cases and samples.

Second, the system should be safe and user controlled. Generated migration SQL should not be executed automatically. Users should be able to review and edit the generated scripts before accepting them. This is important because automatic schema changes are risky, and the client

clearly wanted a safer review-based flow. So, one of our quality expectations is that no destructive action happens without an explicit user step.

Third, the system should be understandable to the target users. Even though the users are technical, not all of them are expected to be strong in databases. Because of that, the interface should clearly show what is missing from each schema, what the version gap is, and what script is being proposed. Our quality expectation here is that key comparison results are shown clearly enough that a user can understand the output without reading raw metadata only.

Fourth, the system should be traceable and auditable. Accepted migration scripts should have a version, and their execution history should be recorded. This matters because one big problem in the current situation is that changes are forgotten or not tracked properly. So, our quality target is that approved changes leave a clear record, at least within the scope of our implemented features.

Fifth, the system should have basic reliability and validation. The app should validate connection input, handle invalid ports or unreachable hosts, and fail safely instead of crashing. It should handle cases where a version table does not exist and go back to structural comparison, as mentioned earlier.

Measurable targets

At this stage, we think these are reasonable measurable targets for our project:

1. Core schema comparison should correctly detect expected differences in our prepared test cases.
2. All main pages should be loaded without blocking errors during demo use.
3. Connection form validation should reject obviously invalid input before submission.
4. Generated migration SQL must always require manual review before acceptance.
5. Accepted scripts should include a recorded version number and timestamp.
6. Main prototype pages should have at least basic tests before the code review stage, in line with the compliance guidance.

These are not perfect industry level measurements, but they are concrete enough for our stage and they match what we are actually building.

Steps to assure quality

To reach those quality goals, we plan to do quality assurance throughout the project, not only at the end.

We will start with requirement validation. Since this project is very client driven, we need to keep checking that our interpretation of the core functions still matches what the client wants. We already learned from the meeting that schema comparison and version control are the main beginning modules, while query analysis and performance features are more secondary for now. So regular requirement checking is part of quality for us, not just coding.

We will also use step-by-step development. Instead of trying to build the whole system at once, we will develop the project feature by feature. For example, first schema comparison, then SQL generation, then version tracking, then GitHub integration. This reduces the chance of building too much on top of a weak base.

For testing, we plan several levels of checking. We will do manual testing on each feature while building, especially for user flows such as sign in, adding connections, choosing schemas, running comparisons, reviewing generated SQL, and viewing versions. We will also prepare sample PostgreSQL schemas with controlled differences so we can test whether the system detects the correct mismatches. For example, one test database may have an extra column, another may have a different data type, and another may be missing a table. This gives us repeatable test data for comparison logic.

We also plan basic integration testing of the full workflow. That means checking whether the frontend, backend logic, application database, and external target database connections work properly together. This is especially important because our project uses several linked tools and not just one standalone page.

On top of that, we will add Jest unit tests for main webpages and key UI behaviour, since this is requested in the compliance notes. Early on, those tests may focus on page rendering, button visibility, form behaviour, and display of comparison results. Later, if time allows, we may extend testing to some logic level functions as well. [08]

Code review within the team will also be part of quality assurance. Before major features are merged, we want at least one teammate to look through the code, especially for risky parts like SQL generation and script versioning. This is not just to catch bugs, but also to keep the code understandable for all three of us.

Definition of done for major backlog items

For us, a feature should not be considered done just because some code exists. A major item is done only when these points are met:

1. The feature works for the user flow.
2. Basic validation and error handling are working.
3. The feature is tested manually on our sample cases.
4. The UI output is understandable enough for use.
5. The code is pushed to the repository and reviewed by the team.
6. If it is a user-facing page, a basic Jest test is added where reasonable.
7. The feature is documented in brief so the rest of the team understands what was implemented.

This is especially important for our project because there is a real chance the plan may shift later depending on difficulty. If we keep a clear definition of done, we can at least be sure that what we say is completed is stable enough to build on.

Final note on quality planning

We want to be honest that this quality plan is still based on our current understanding. Some parts will probably need adjustment once we get deeper into implementation, especially around migration rollback, GitHub automation, and version classification rules. But our overall direction is clear. We want a system that is correct in the core workflow, safe in how it handles schema

changes, understandable for its users, and tested enough that we can trust our demo and build on it properly.

8. RESEARCH AND INVESTIGATIONS

We conduct research during the preparation of this project. with the goal of obtaining knowledge and information to be able to create or decide about the business domain, existing tools, architectural patterns, schema comparison algorithms, performance monitoring Business Domain Understanding.

This project addresses the challenge of managing database and schema changes across various environments (development, staging, production). In modern software development, manual schema synchronisation often leads to inconsistencies, delayed deployment, and lack of auditability. this project also helps keep version control.

As mandated in the Application Support Compliance document, a requirement study was conducted. The initial specifications were provided via email from the client, followed by a meeting where the requirements were explained in further detail. The following are the technology stack.

- Framework: NextJS
- Database: PostgreSQL with Sequelize
- Authentication: Microsoft Account (OAuth) Integration with NextAuth
- Communication: HTTP Fetch or Axios with clear UI and API separation
- Styling: Tailwind CSS (desirable) or standard CSS
- Deployment: Containerized via Docker
- Jest unit testing for all webpages (additional requirement)

The use of AI was highly desirable for helping to show the difference between schema and highlight critical or minor changes to help prioritized update, generate sql script Generation, Query Execution Analysis, Performance Suggestion, Performance Monitoring and Migration Version Validation & Deployment.

Research on Existing Systems

Here are few examples of existing system to identify best practices

Tools	Key Features
Liquibase	Using both declarative and imperative methods, an opensource command-line tool for tracking, versioning, and deploying database changes.
Bytebase	an open-source database DevOps solution that manages databases during the software development lifecycle (SDLC) using GitLab/GitHub.
dbForge Studio	It allows you to compare and synchronize database schema between databases and SQL files. It has two distinct futures: Diff and synchronize MySQL databases quickly and efficiently, even for extra-large ones. Automate routine tasks via CLI to schedule your comparison and synchronization tasks.
VSCoDe (MSSQL Extension)	compare schemas between files database and SQL project, view results.

Table 6: Existing system

Synthesis:

Research shows that there are tools that offer similar capabilities, but no single tool combines all desired capabilities.

UNIT CODE COS40005
PORTFOLIO TASK 1

Specialized Research Topics

Schema Comparison Algorithms:

Information about the tables structures, columns, indexes and constraints was retrieved through research into Pg System catalogs (`pg_class`, `pg_attribute`, `pg_index`, `pg_constraint`). In the end, a two-phase engine was implemented: first version detection (when there's a version table in place), second structural comparison.

SQL Script Generation and Versioning:

Best practices of Liquibase and Flyway were discovered for creating both a migration (up) script and rollback (down) script. Semantic versioning (x.y.z) based on breaking (major), additive (minor), or minor fixes (patch). Scripts are organized in a version-controlled Git repository located at the `/scripts_init` and `/scripts_patch` folders.

Query Analysis and Performance Monitoring:

In addition to examining PostgreSQL's performance views (`pg_stat_statements`, `pg_stat_activity`, `pg_statio_*`) and EXPLAIN plan parsing for sequential scans, missing indexes, and expensive operations. We explored integrating AI (e.g. OpenAI API) to convert EXPLAIN output into plain-language recommendations. Performance metrics will be sampled at intervals, to avoid the overhead of capturing many [key: value] pairs every second/minute/period. These performance snapshots would then be stored on their own table.

Security and Access Control:

Research on securing database connection strings led to the use of environment variables and Docker secrets. Role-based access control (RBAC) is handled in NextAuth callbacks that map Microsoft Entra ID groups to application roles (e.g., "Schema Admin", "Read-Only User").

Synthesis and Justification

The research directly informed the system design:

Next.js + Sequelize – mandated by client but adhere to industry good practices for full-stack JavaScript applications.

This is a two-phase comparison engine based on what we learned by rifling through PostgreSQL introspection and how version tables are used in existing tools.

Semantic version — borrowed from Liquibase/Flyway for better change impact communication.

Docker containerization – studied to ensure environment uniformity and facilitate deployment.

Jest unit testing – Normalcy, guarantees stability and allows code review.

And such decisions consider the client’s compliance requirements along with proven technical practices to strike a balance in a solution that is maintainable and effective.

9. KOST ANALYSIS

Skill	Capability	Gap	Plan to overcome
NextJS	Have some basic knowledge	Limited experience with APi, SSR, and deployment.	Complete Next.js tutorials, try and practice using API routes and SSR. Consult supervisor.
PostgreSQL with Sequelize	Know some SQL knowledge	No prior experience with an ORM (Sequelize) or advanced	Learn and view some tutorials and consult with supervisors.
		PostgreSQL features (system catalogs, performance views).	

Microsoft Account (OAuth) Integration with NextAuth	Have some knowledge	No hands-on experience.	Follow the provided guide.
Tailwind CSS (desirable) or standard CSS	Know basic CSS	No experience with Tailwind.	Complete tailwind CSS tutorial try to apply for it.
Containerized via Docker	No prior experience	Zero knowledge of containerization.	View tutorial for docker.
Jest unit testing	Have experience in unit testing	Need to adapt existing skill on Next.js.	Review jest and react testing library.
AI	Have a little experience	No practical experience.	Experience more, learn more information.

Table 7: Kost analysis

10. ETHICAL CONSIDERATIONS

A. Privacy Law

Australia Privacy Act and Australian Privacy Principles (APPs) as Swinburne University is affiliated with Australia university in Malaysia, The Personal Data Protection Act 2010 (PDPA)

UNIT CODE COS40005
PORTFOLIO TASK 1

because we are operating in Malaysia and the database might content Malaysian individual data. The system only collects necessary data, and the data collected by user with consent and protects user privacy as per laws.

B. Data Security

Leak of confidential data and personal data can damage trust and compromise the databases. The project would be used to compare various kinds of data including sensitive data such as security measures are essential database can be encrypted using environment variable access to credentials will be restricted to authorized processes only.

C. Access Control

Data should be controlled with access to reduce data loss, corruption and unauthorized changes to mitigate role base access control this prevent users who are not supposed to be able to change data still able to access (View/read) the data itself reducing he change unauthorized individual to read data without the power of changing it.

D. AI ethics

AI is used to explain schema differences, generate SQL script suggestion. safeguard needed Users must review and approve all AI- generated recommendations before the action is taken. Users can control that the AI is Optional.

Reference list

Candy (2026). *Top 4 MySQL Schema Compare Tool to Diff and Sync Database 2026*.

[online] Bytebase. Available at: [https://www.bytebase.com/blog/top-mysql-schema-compare-tools/compare-tools/](https://www.bytebase.com/blog/top-mysql-schemahttps://www.bytebase.com/blog/top-mysql-schema-compare-tools/compare-tools/)

[Accessed 28 Mar. 2026].

Docker (2019). *Docker Documentation*. [online] Docker Documentation. Available at:

<https://docs.docker.com/>.

Flywaydb.org. (2021). *Redgate Flyway Documentation - Redgate Flyway - Product Documentation*. [online] Available at: <https://flywaydb.org/documentation/>.

Liquibase.com. (2025). Available at: <https://docs.liquibase.com/>.

next-auth.js.org. (n.d.). *NextAuth.js*. [online] Available at: <https://next-auth.js.org/>.

nextjs.org. (n.d.). *API Routes: Introduction | Next.js*. [online] Available at: <https://nextjs.org/docs/api-routes/introduction>.

PostgreSQL Documentation. (2026). *Chapter 52. System Catalogs*. [online] Available at: <https://www.postgresql.org/docs/current/catalogs.html> [Accessed 28 Mar. 2026].

rwestMSFT (2026). *Overview of the MSSQL Extension for Visual Studio Code - SQL Server*. [online] Microsoft.com. Available at: <https://docs.microsoft.com/en-us/sql/tools/visual-studio-code/mssql-extensions> [Accessed 28 Mar. 2026].

sequelize.org. (n.d.). *Sequelize v6 | Sequelize*. [online] Available at: <https://sequelize.org/docs/v6/>.

Siddique, T. (2025). *MSSQL Extension for VS Code: Introducing Schema Compare (Preview) - Azure SQL Dev Corner*. [online] Azure SQL Dev Corner. Available at: <https://devblogs.microsoft.com/azure-sql/vscode-mssql-schema-compare/> [Accessed 28 Mar. 2026].

TrustCloud. (2025). *Ethical AI & data privacy best practices | Governance guide for 2026*. [online] Available at: <https://www.trustcloud.ai/ai/boost-trust-with-powerful-ethical-ai-and-data-privacy-practices/ethical-ai-and-data-privacy-practices/#what-are-data-privacy-and-ethical-considerations> [Accessed 28 Mar. 2026].

UNESCO (n.d.). *Ethics of Artificial Intelligence*. [online] UNESCO. Available at:
<https://www.unesco.org/en/artificial-intelligence/recommendation-ethics>.